

A Benchmarking Study of Multi-Layer Security Filters for Secure RAG Pipelines

Nidhi Mishra,
Dept. of Computer Science &
Engineering, Amity School of
Engineering & Technology,
Amity University
Noida, India

<https://orcid.org/0000-0001-5881-1955>

Kanchan Rai,
Dept. of Computer Science &
Engineering, Amity School of
Engineering & Technology,
Amity University
Noida, India

kanchanrai2307@gmail.com

Dhruv Sharma,
Dept. of Computer Science &
Engineering, Amity School of
Engineering & Technology,
Amity University
Noida, India

dhruv0001638@gmail.com

Abstract—This study explores how to protect sensitive information when using retrieval augmented generation systems that connect large language models to external databases. RAG technology has become popular for enhancing language model outputs by linking them to specialized knowledge repositories, which produces more accurate and grounded responses. Despite these benefits, such systems pose privacy risks because they may inadvertently expose confidential data stored in their connected databases. Existing research proposes a security solution built around multiple layers of filters and their current approach implements three different filtering stages that work together to identify and remove sensitive content which a language model may generate. The system was tested with seven prominent language models where real world data leak scenarios were designed and mimicked and the results demonstrated that this system substantially decreased the likelihood of sensitive information leak while maintaining quality of the system's responses.

Keywords—*Retrieval Augmented Generation, Adversarial Attacks, Large Language Models, Information Security, Prompt Injections, Data Leakage Prevention*

I. INTRODUCTION

Currently almost every Large Language Model delivers high quality performance across various fields, their real-world utility is often boosted by Retrieval Augmented Generation. This particular approach improves both factual accuracy and domain specific relevance. By pointing model outputs in external data repositories these pipelines successfully minimize hallucinations and ensure logical consistency.

The integration of third-party information or domain specific information often comes with trade-offs which broadens the overall attack surface and introduces more privacy and security risks. These include a higher susceptibility to adversarial manipulation and the danger of accidental data leaks [1].

During the retrieval phase if data brings a private or sensitive detail, the system faces a risk of exposure. This can happen through standard user interactions or malicious prompts. Such flaws open the possibility of data extraction putting sensitive and confidential information, files, and legally protected information at risk [2]. These breaches often fail to meet fundamental standards for data sanitization.

Currently many companies rely on standard defences like manual oversight, pattern matching, or access controls, these tools fall short when facing a modern attack. LLMs are notoriously poor at spotting and stopping security threats on their own, especially when those threats have deceptive prompt engineering [3]. Systems using RAG are particularly prone to prompt injection. In this case commands often override current safety rules letting attackers ignore the retrieved facts or sidestep security protocols entirely [4].

Available fixes such as encrypting databases or filtering output after it is generated offer only limited protection. These methods completely fail to solve the main problem that is that language models do not truly understand the security logic or the meaning of sensitive information. Only because of this reason finding a balance between keeping data private and ensuring system resilience is a hurdle for deploying RAG frameworks safely [17].

The following research introduces a modified type of RAG system which is designed to tackle security gaps by combining pipeline wide safety features with security conscious distilled models. The architecture has a pre-inference filter to clean the data before the model ever sees it. It also uses real-time protections during the generation to block the leakage of sensitive information. Artificial datasets were built to test the system thoroughly while keeping the actual data safe. These datasets included sectors like finance, medical care, law, and public services.

The following work focuses on analysing RAG flaws, deploying a multi-tier content defence and building a distilled model tuned specifically for security. The whole system was rigorously tested against the hostile inputs. Even after that finding the right balance between protecting privacy and ensuring system strength and providing accurate answers remains a major obstacle for actual RAG deployments [5].

Studies conducted in the earlier years suggested that security holes often appear as early as the initial data intake and clean-up phases [7]. Researchers have also shown that bad actors can mess with retrieval order by injecting custom documents. This allows them to hide the

correct context [9]. Risks to privacy also show up at the representation level as here the mathematical embeddings and retrieval tools might accidentally reveal secret data [15, 19].

II. LITERATURE REVIEW

A comprehensive survey conducted by Zhao et al. emphasizes the significance of effective knowledge integration from external sources, while overcoming problems such as data privacy, robustness, and reliability, etc. [2]. Zhou et al. also emphasize trustworthiness in RAG systems, with one major issue being identified as “privacy leakage” during both retrieval and generation phases [17].

A few studies have been conducted regarding risks due to privacy issues, data leakage, etc. In this context, Zeng et al. have shown how adversarial prompts can be used to leak sensitive information from retrieval databases, thus creating problems for RAG systems [7]. Di Maio et al. have further shown how attackers can use RAG systems to leak knowledge bases using adaptive attack methods [1]. RoyChowdhury et al. have also discussed “confused deputy” risks, in which restricted information can be inadvertently disclosed [10].

Advanced attack methods have also been considered. Shafran et al. describe the concept of jamming attacks, which affect the retrieval process, thereby impairing the system's performance [9]. Peng et al. describe data extraction attacks via the backdoor approach, emphasizing the possibility of unknown vulnerabilities in the data used during the retrieval process [16]. Anderson et al. describe membership inference attacks, focusing on the possibility of determining the presence of specific data in the retrieval database [20].

In this context, several defense mechanisms have been proposed to address these problems. For instance, Yao et al. proposed the ControlNet, which is essentially a firewall-like architecture for securing RAG systems against malicious queries [5]. Another such mechanism has been proposed by Nandagopal, which offers an exhaustive secure pipeline framework for securing RAG systems [3]. Zhou et al. have proposed several privacy-aware RAG models for securing knowledge retrieval processes [14].

Another promising direction for addressing RAG security and privacy concerns is the use of synthetic data, as discussed in several recent studies. For instance, Zeng et al. have shown that synthetic data can be effectively used to minimize several RAG-based privacy risks while ensuring system performance [22]. In this context, Wang et al. have proposed several privacy-aware decoding techniques for addressing information leakage during text generation [21].

From the above discussion, it can be inferred that RAG systems, while offering several advantages for enhancing LLMs, pose critical security and privacy risks, such as data extraction, prompt injection, and inference attacks, which can be addressed through several defense mechanisms.

Having a few successful attempts in protecting RAG environments the current defensive frameworks are still

frequently tested in a vacuum. Which means they lack the deep, comparative analysis needed to see how they perform across various model designs or evolving threats. A large amount of existing theories focus on some very narrow setups usually involving just one particular language model or a handful of attack types which makes it more hard to implement in a real world scenario [8].

The effectiveness of these security measures are often shared using inconsistent or informal metrics which focus on standardized benchmarks such as success rates or a balance between the content filtering and the actual performance of the system. This broken approach proves that more unified testing methods are required, such strategies must be able to track the relationship between security, reliability, and speed within an actual standard RAG installation to be truly useful.

III. METHODOLOGY

A. Overview of the Secure RAG Framework

The given methodology presents a systematic way to protect a RAG based pipeline by layering multiple security protocols throughout the data related processes. Instead of relying on a single defensive gate, the given framework uses a defence in depth approach. This reduces the chances of private data leaking out at plenty of high-risk points specifically during the pre-processing, inference phase and the final generation step.

By focusing particularly on the data serving portion of the LLM lifecycle the framework prioritizes the rigorous data cleaning and highly regulated retrieval steps which result in a fortified output production. The system is built to be model agnostic on purpose. This allows it to work seamlessly with various types of language models whether they are generative or extractive. A design like this ensures that security testing remains consistent across different setups without the need for custom changes.

B. Threat Model and Design Assumptions

By utilising an honest but curious type of threat model here we assume that the attacker will use an official query box and will follow the rules but is secretly trying to squeeze out extra sensitive information from the system's responses. We assume they cannot get into the actual vector database, or the internal model weights or the main libraries.

Instead of that an attacker is just really good at writing sneaky prompts to trick the system into leaking sensitive data during the retrieval or generation phase. In this setup there are either direct or indirect data leaks, prompt injection and various social engineering tricks. The main goal of the system is to stop the sensitive information from getting out without ruining the quality of the answers or making the system too slow.

C. Multi-Layer Filtering Architecture

They are intended to block the data leaks throughout the execution flow by distributing them across

several phases. It ensures if one specific control is evaded or fails, then it does not lead to the immediate disclosure of

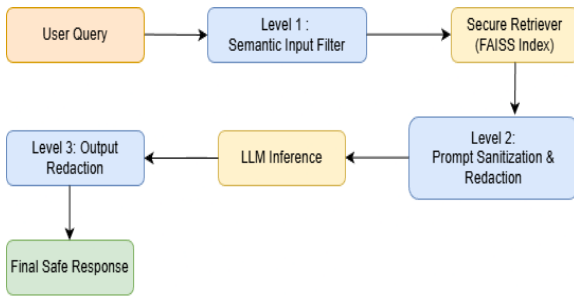


Figure 1. Pipeline Architecture

private information which significantly bolsters the durability of the whole system. As shown in Figure 1 it provides a broad overview of the pipeline structure and demonstrates the way these specialized layers work during active use.

Every question or query from the user is first turned into a mathematical format called semantic representation which allows the system to run a cosine-based similarity test. It then compares the new question or query against a database of known "bad" queries or topics. If the similarity score is too high the system realizes it's likely a prompt injection or a data theft attempt and kills the request before it even gets to the retrieval stage. This is a big deal because it stops attackers at the very front door long before they can interact with any sensitive data.

In the background the knowledge library is also being protected but before anything is even indexed a pre-indexing cleanup takes place. It uses entity recognition to find and strip out private details like names, social security numbers, bank information or other relevant sensitive information. This is a critical step because it means the system only ever creates vector embeddings from safe versions of the data. Since the raw sensitive information never makes it into the vector database in the first place an attacker can't steal even if they manage to trick the retrieval process into pulling a specific document.

As a last safeguard a final output review layer is there. It scans the generated responses for residual sensitive information which may arise from model hallucinations. Together with the input and retrieval defense this layered design provides a kind of failsafe mechanism in which overlapping protections catch attacks that evade individual filters making the architecture significantly more robust than single stage defenses.

D. Synthetic Data Generation and Knowledge Base Construction

For a more ethical and consistent testing process a bunch of synthetic datasets were developed covering various fields including medical care, banking, public services and online retail. These data collections are built using automated techniques which produce almost human-like records which are then saved in a structured JSONL format to handle large scale processing. The given

configuration allows private fields as well, for testing purposes.

The private fields are stripped away during the cleanup phase before they ever reach the index. By utilizing the artificial data, the given setup eliminates the danger of revealing actual private information while still maintaining the level of detail needed for a deep analysis or audit [22]. This ensures that the system is tested against real life scenarios without compromising privacy.

E. Evaluation Methodology and Metrics

The given setup's effectiveness undergoes a systematic testing through adversarial cases divided into five groups, firstly data leakage then resistance to deception followed by harmful content generation, instructional vulnerability and social engineering. Primarily the performance was measured using Attack Success Rate which tracks how often data theft attempts actually work.

The test cases have been divided into 5 groups. Data leakage and deception resilience have 5 prompts per model each whereas harmful content generation has 4 prompts per model. Instructional vulnerability and social engineering have 3 prompts each. The total test cases turn out to be 20.

To provide a clearer picture of the defensive power, Security Score is also calculated as the inverse of Attack Success Rate. Latency metrics are also tracked to determine the extra processing time and performance costs caused by multi layered filter systems which allows us to see the exact trade-off between keeping the system safe and fast.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

The current setup was experimentally studied to methodically test how well the multi-layer filtering works to protect RAG based pipelines from sophisticated adversarial attacks. While maintaining ethical standards and allowing for a more controlled and repeatable study of data leakage the research utilized exclusive synthetic datasets. These datasets cover six vital fields including medical care, finance, public services, law, schooling and online retail. RAG systems often handle private data in the above-mentioned areas. Furthermore, the hostile scenarios were also designed to imitate actual RAG based attacks making sure the results are accurately represented and can resemble actual dangers in the real or production level setup [18].

The secure RAG design combines three defensive layers which are synchronized with each other. Query level checks at the start, followed by data cleaning in the retrieval phase and lastly responsive verification at the final step. Before indexing even begins every document experiences a thorough cleaning process. For this context sensitive entity recognition is used to find and strip out the private markers like financial codes, contact details, personal identifiers etc. By only turning this sanitized answer the last validation

check inspects the text to ensure no sensitive information is revealed before the user sees it.

In experimental testing a mix of different language models, ranging from massive generative systems to smaller ones. It was kept to analyze and observe how the framework held up across a broad spectrum. To allow other users to test the same setup every model was put through the exact same ringer. Every model had the same datasets, retrieval settings and the same set of malicious prompts. They were kept consistent to get a reliable comparison of how these different architectures actually handle pressure.

Malicious prompts were designed to cover a wide variety of attack styles, from direct jailbreak attempts to subtle and simple prompt injections, everything was thoroughly tested. Many sets of reference questions and answers were used to see exactly where a model started to drift or fail. It was important when compared to larger models against the smaller ones. By keeping the configuration the same for every run it was ensured experimentally that any change in Attack Success Rate was because of the model's architecture.

B. Experiment Evaluation

The evaluated results show how the multi-layer security frameworks actually perform across different large language models. Comparison of secured setups to basic and unfiltered versions standard tests were used to track both safety and speed. The main metric was the ASR or Attack Success Rate which tracks how often a hostile input breaks through a model's own guardrails and the given extra security layers. Security Score was also used to make and present the evaluation in a much simpler way. Security Score is just the inverse of the ASR. If the ASR is high it means that the model is wide open to exploitation.

The Attack Success Rate results and Security Score results are summarized in Table I. Looking at the baseline models without any protection setup they were highly vulnerable to data theft in almost every design. Models like Mistral Large 3, Gemini 2.5 Flash, Groq, MiniLM and BERT-QA were completely susceptible to these attacks. In a few scenarios these models didn't even try to avoid the prompt injection. The above results prove that a model's internal safety cannot be relied on directly while using RAG, especially in the fields with strict rules. The guardrails are often generalized to catch a very specific or targeted attack on a retrieval pipeline.

When the system was set up with multi-layer filtering the ASR dropped massively for every model that was tested. Largest generative models had the biggest safety boost because of their built-in advanced ability to understand context. It worked well with input and output filters as they understood the intent of security layers. In smaller extractive models' overall vulnerability was there but the security was much more improved. It is because they are not able to follow a specific instruction to refuse a harmful request because of a lack of complex reasoning.

After looking at the results, the given setup is efficient at stopping direct data leaks, bypassing system

instructions and generating harmful content. But at the very same time with social engineering it can still miss some threats as shown in Table II. Smaller models often do not struggle here because they don't have the reasoning depth to spot a malicious intent hidden inside a prompt. It is clearly demonstrated that a RAG based system needs more than just a generic filter; the various attack success rates for the various domains have been shown in Figure 2. A need for layered defense is clearly missing. With the help of custom layers different models can handle safety rules really well.

Apart from security the experimental evaluation also checked the technical footprint to see if this setup works for a production level setup. Latency was measured as the total time from query submission to the final response which included the retrieval, the three filtering stages and the model inference. In the basic setup the wait time is mostly the model's own time. Naturally, the bigger models take longer to think.

When the security layers are active, the extra delay is only about 0.27 seconds which is very small and stays consistent across different architectures. In some cases, the system was found to be more efficient. By catching bad queries at the very start it can kill the request before it even gets to the expensive steps like vector search. The fail fast approach here saves both the time and computing power of a model. BERT-QA was a bit of an outlier here, with slightly higher overhead because of its specific focus on span-based inference. The results in Table III show the framework gives a major security boost with almost no impact on speed making it a good fit for low latency RAG apps. To calculate and evaluate how well the given system stood up to the input we use Attack Success Rate as our main metric.

TABLE I. AGGREGATED ASR AND SECURITY SCORE ACROSS MODELS

Model Name	ASR Unfiltered (%)	Security Score Unfiltered (%)	ASR Filtered (%)	Security Score Filtered (%)
Mistral Large 3	100	0	0	100
Gemini 2.5 Flash	100	0	10	90
Groq	100	0	10	90
LlaMA3-70b Instruct	90	10	20	80
DeepSeek	75	25	25	75
MiniLm	100	0	60	40
BERT-QA	100	0	85	15

TABLE II. ATTACK SUCCESS RATES (%) ACROSS SECURITY DIMENSIONS (FILTERED CONFIGURATION)

Model Name	Data Leakage	Deception	Harmful	Instructional	Social Engineer
------------	--------------	-----------	---------	---------------	-----------------

		Resilience	Content	Vulnerability	ing
Mistral Large 3	0	0	0	0	0
Gemini 2.5 Flash	0	0	0	0	33.3
Groq	20	20	0	0	0
LlaMA3-70b Instruct	20	40	25	0	0
DeepSeek	20	40	50	33.2	0
MiniLM	0	100	100	0	100
BERT-QA	100	100	100	0	100

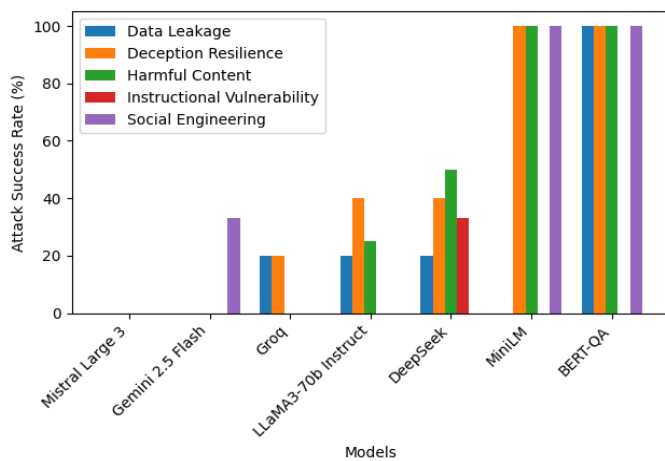


Figure 2. Attack success rate across security domains

In fig 2 ASR tracks the frequency at which a hostile prompt or a test case manages to steal the sensitive information by breaking through both internal safety rules of a model and multi-stage security layers.

Since it measures these outcomes directly, ASR provides a clear answer of how vulnerable the system is when facing an actual attack. A lower ASR indicates that the pipeline is more resilient and a high percentage highlights specific gaps where the defensive layers could be improved.

ASR as a metric extends beyond a simple assessment as it can be segmented across different categories. By assessing ASR across various categories like prompt complexity, attack sophistication level or specific types of sensitive information targeted security teams can prioritize their hardening efforts on the most vulnerable spot. This approach to measure attack success enables organizations to check on their defences systematically tracking improvements over time and validating that security enhancements actually reduce the

TABLE III. LATENCY OVERHEAD ADDED IN FILTERED CONFIGURATION

Model Name	Baseline latency(s)	Filtered latency(s)	Added Overhead(s)
Mistral Large 3	0.0206	0.2906	+0.27
Gemini 2.5 Flash	0.6249	0.8949	+0.27
Groq	0.1958	0.4658	+0.27
LlaMA3-70b Instruct	3.5997	3.8697	+0.27
DeepSeek	1.7876	2.0576	+0.27
MiniLm	0.0196	0.2896	+0.27
BERT-QA	2.7437	3.0137	+0.27

attack surface rather than merely shifting vulnerabilities to different areas of the system.

$$ASR = \frac{\text{Number of Failed Test}}{\text{Total Number of Tests Conducted}} \times 100 \quad (1)$$

$$\text{Security Score} = 100\% - ASR \quad (2)$$

V. CONCLUSION & FUTURE SCOPE

The given study tests a layered security setup to stop RAG based pipelines from leaking sensitive information. It checks all the questions at the start, cleans them in the middle and then double checks the final answer. Because of these extra layers it keeps the sensitive information from slipping out while the system is running. When tested with synthetic data, the given setup made a huge difference as compared to standard RAG setup. The biggest AI models improved but there were still some social engineering-based trick questions which were hard to block entirely. The given study tells us that the security has to be built right into the pipeline from the start especially for sensitive and important domains such as healthcare or banking. This study evaluated a layered security approach for RAG based pipelines that filters queries, sanitizes the output and reviews the output to prevent data leakage. As per the results the standard RAG systems turned out to be highly vulnerable but when multiple security layers were added it significantly improved protection though social engineering attacks remain challenging. The current findings emphasize on the need for built in pipeline security in sensitive domains. Moving forward the goal is to catch bad queries even earlier and see how much these layers slow things down when the system gets busy. It also includes broader model evaluation as the technology keeps changing.

REFERENCES

- [1] Di Maio, C., Cosci, C., Maggini, M., Poggioni, V., and Melacci, S., "Pirates of the RAG: Adaptively Attacking LLMs to Leak Knowledge Bases", arXiv:2412.18295v1 [cs.AI], Dec. 2024.
- [2] Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., and Qiu, L., "Retrieval Augmented Generation (RAG) and Beyond: A Comprehensive Survey on How to Make your LLMs use External Data More Wisely", arXiv:2409.14924v1 [cs.CL], Sep. 2024.

- [3] Nandagopal, S., "Securing Retrieval-Augmented Generation Pipelines: A Comprehensive Framework", *Journal of Computer Science and Technology Studies*, 2025.
- [4] Zhu, Z., Shao, N., Lian, D., Wu, C., Liu, Z., Yang, Y., and Chen, E., "Understanding Privacy Risks of Embeddings Induced by Large Language Models", *Proceedings of ACM Conference (Conference '17)*, 2024.
- [5] Yao, H., Shi, H., Chen, Y., Jiang, Y., Wang, C., and Qin, Z., "ControlNet: A Firewall for RAG-based LLM System", *arXiv:2504.09593v2 [cs.CR]*, Apr. 2025.
- [6] Castagnaro, A., Salviati, U., Conti, M., Pajola, L., and Pizzi, S., "The Hidden Threat in Plain Text: Attacking RAG Data Loaders", *arXiv:2507.05093v1 [cs.CR]*, Jul. 2025.
- [7] Zeng, S., Zhang, J., He, P., Xing, Y., Liu, Y., Xu, H., Ren, J., Wang, S., Yin, D., Chang, Y., and Tang, J., "The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)", *arXiv:2402.16893v1 [cs.CR]*, Feb. 2024.
- [8] Dong, Y., Mu, R., Zhang, Y., Sun, S., Zhang, T., Wu, C., Jin, G., Qi, Y., Hu, J., Meng, J., Bensalem, S., and Huang, X., "Safeguarding Large Language Models: A Survey", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [9] Shafran, A., Schuster, R., and Shmatikov, V., "Machine Against the RAG: Jamming Retrieval-Augmented Generation with Blocker Documents", 2025.
- [10] RoyChowdhury, A., Luo, M., Sahu, P., Banerjee, S., and Tiwari, M., "ConfusedPilot: Confused Deputy Risks in RAG-based LLMs", *arXiv:2408.04870v5 [cs.CR]*, Oct. 2024.
- [11] Anonymous Authors, "DEAL: HIGH-EFFICACY PRIVACY ATTACK ON RETRIEVAL-AUGMENTED GENERATION SYSTEMS VIA LLM OPTIMIZER", Under review at ICLR 2025.
- [12] Zhou, X., et al., "A Survey of LLM $\times$ DATA", *arXiv:2505.18458v3 [cs.DB]*, Jun. 2025.
- [13] Du, X., et al., "Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG", *arXiv:2406.11147v3 [cs.SE]*, Jun. 2025.
- [14] Zhou, P., Feng, Y., and Yang, Z., "Privacy-Aware RAG: Secure and Isolated Knowledge Retrieval", *arXiv:2503.15548v1 [cs.CR]*, Mar. 2025.
- [15] Li, X., et al., "TARGETING THE CORE: A SIMPLE AND EFFECTIVE METHOD TO ATTACK RAG-BASED AGENTS VIA DIRECT LLM MANIPULATION", *arXiv:2412.04415v1 [cs.AI]*, Dec. 2024.
- [16] Peng, Y., et al., "Data Extraction Attacks in Retrieval-Augmented Generation via Backdoors", *arXiv:2411.01705v2 [cs.CR]*, Mar. 2025.
- [17] Zhou, Y., et al., "Trustworthiness in Retrieval-Augmented Generation Systems: A Survey", 2024.
- [18] Xue, J., et al., "BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models", *arXiv:2406.00083v2 [cs.CR]*, Jun. 2024.
- [19] Liu, J., et al., "Exposing Privacy Risks in Graph Retrieval-Augmented Generation", *arXiv:2508.17222v1 [cs.CR]*, Aug. 2025.
- [20] Anderson, M., et al., "Is Myoku Data in Your Retrieval Database? Membership Inference Attacks Against Retrieval Augmented Generation", *arXiv:2405.20446v3 [cs.CR]*, Feb. 2025.
- [21] Wang H, Xu X, Huang B, Shu K. Privacy-aware decoding: Mitigating privacy leakage of large language models in retrieval-augmented generation. *arXiv preprint arXiv:2508.03098*. 2025 Aug 5.
- [22] Shenglai Zeng, Jiankun Zhang, Pengfei He, Jie Ren, Tianqi Zheng, Hanqing Lu, Han Xu, Hui Liu, Yue Xing, and Jiliang Tang. 2025. Mitigating the Privacy Issues in Retrieval-Augmented Generation (RAG) via Pure Synthetic Data. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24538–24569, Suzhou, China. Association for Computational Linguistics.